

Architectural Blueprint for Data-Driven Progression Maps: Constraint-Based Graph Visualization and High-Performance UX

I. Conceptualizing the Progression Map: Beyond the Skill Tree

1.1. The Progression Metaphor: From Linear Trees to Explorable Worlds

Game progression systems are critical infrastructure for achieving player retention and engagement. They establish a fundamental cycle of goal completion, reward dissemination, and forward momentum, compelling players to continue their journey through increasing challenge and competence.¹ Historically, this system has been visualized through the ubiquitous "Skill Tree," a system popularized by early RPGs like *Diablo 2*.²

However, the traditional skill tree often proves limiting. Many designers have implemented systems that feel artificial or generic, containing abilities that could have been integrated from the start, thus frustrating users and diminishing the sense of meaningful advancement.² The progression system must communicate more than mere prerequisite linkage; it must convey a sense of **spatial coherence, historical record, and achievement**.

The industry trend demonstrates a clear move toward spatially rich progression models. For example, in the transition from *Path of Exile 1* to *Path of Exile 2*, the system evolved from an abstract concept to a "cleaner and richer world design," including an "act map" and a "3D atlas" that successfully provides a "real sense of progression".³ This architectural shift from an abstract graph diagram to an explorable map implies that the visualization must function as a

persistent topological record of past efforts, not just a list of future requirements. This aesthetic necessity elevates the underlying engineering challenge from simple graph drawing to high-performance, **constrained topological mapping**, where the layout must remain stable over time and respect predetermined spatial relationships.

The structural requirement for the progression visualization is therefore not merely a presentation layer but a critical component of the user experience.

1.2. Defining Data Structures and Domain Constraints

The progression map inherently represents a series of goals, challenges, or events (nodes) connected by prerequisites or paths (edges). This relationship structure dictates that the map must fundamentally function as a **Directed Acyclic Graph (DAG)**, where the flow (edges) moves consistently in one direction (e.g., outwards from the starting point, or downwards through layers of increasing difficulty).⁴

In this specific domain, the data mapping assigns:

- **Nodes:** Represent skills, major achievements, events, or entire biomes/zones.
- **Edges:** Represent prerequisites, unlocking paths, or dependencies (directional flow).
- **Weights/Metadata:** Can be assigned to nodes or edges to represent cost, experience requirements, time investment, or thematic groupings.

Developing any auto-layout system requires the establishment of constraints because, without these limitations, an "infinite number of possible representations exist".⁶ For a progression map, the constraints are severe and often conflicting: they must satisfy the logical flow (DAG structure) while supporting the visual metaphor (organic, world-like clusters). For instance, if the vertical axis is used to represent a non-negotiable factor like cumulative cost or time progression, the node's Y-coordinate must be fixed by that timestamp.⁷ The flexibility to define these constraints is paramount for achieving a functional and aesthetically viable progression system.

1.3. Establishing Aesthetical Criteria (The "World Map" Vision)

To fulfill the vision of an explorable "World Map," the graph layout algorithm must prioritize aesthetics typically associated with organic clustering and smooth navigation, moving beyond the rigid, orthogonal lines of traditional flowcharts.

The core aesthetic criteria for this implementation, derived from advanced graph drawing principles, include:

1. **Layout Stability:** Minimal rearrangement or "jitter" when a single node is unlocked or a small subset of nodes is added.⁹
2. **Minimal Edge Crossings:** Crucial for visual clarity, especially when paths are dense.¹⁰
3. **Predictable Path Curvature:** Edges should either be uniform in length or follow smooth, aesthetically pleasing curves, rather than arbitrary orthogonal routes.¹⁰
4. **Balanced Distribution:** Nodes must be distributed evenly to avoid overly dense clusters or sparse, empty regions, reflecting principles of repulsive forces separating unconnected nodes.¹⁰
5. **Directionality Emphasis:** The visual flow must clearly indicate forward momentum, even without relying solely on explicit arrows.¹¹

Meeting these criteria requires a sophisticated algorithmic approach that synthesizes the logic of hierarchical drawing with the aesthetic qualities of force simulation, a technique addressed through constraint-based hybrid models.

II. Advanced Graph Layout Architectures for Stability and Flow

The visualization of the progression map presents a classic dilemma in graph drawing: prioritizing logical hierarchy (flow) versus prioritizing aesthetic appeal (organic grouping). No single conventional algorithm satisfies all requirements, necessitating a hybrid, constraint-based approach.

2.1. Hierarchical (Layered) Layout Systems (Sugiyama-Style)

Layered graph drawing, also known as hierarchical layout or the Sugiyama algorithm, excels at visualizing directed dependencies. The method requires assigning graph vertices to horizontal rows or "layers" such that all edges generally flow downwards, emphasizing the hierarchy and directionality.⁴

The layout construction proceeds through three complex stages:

1. **Acyclicity:** If the graph contains cycles, a set of edges must be reversed to ensure a

DAG structure (the NP-complete feedback arc set problem).⁴

2. **Layer Assignment:** Vertices are assigned to layers such that edges move from higher to lower layers.
3. **Crossing Reduction:** The nodes within each layer are sequenced to minimize edge crossings, a critical step that often relies on effective heuristics due to its NP-hard nature.⁴

If the data model dictates that one dimension (e.g., the Y-axis) must correlate to a fixed metric like time or cumulative required experience, the layered drawing technique is structurally necessary.⁷ However, while logically sound for prerequisite management, layered layouts often produce a visualization that is overly schematic and structurally rigid, which can clash with the desired "organic world map" aesthetic.¹³

2.2. Force-Directed Layouts: The Organic Approach

Force-directed algorithms are highly valued in data visualization because they naturally produce aesthetically pleasing, organic layouts that often mirror the intrinsic clustering within the dataset.¹⁴ These methods treat the graph as a physical system: spring-like attractive forces (based on Hooke's law) draw connected nodes together, while repulsive forces (like electrically charged particles based on Coulomb's law) separate all pairs of nodes, resulting in an equilibrium state.¹⁰ This simulation is highly effective at revealing dense, connected regions.¹⁵

Despite their aesthetic appeal, traditional force-directed layouts suffer from two significant limitations in the context of a progression map:

1. **Lack of Stability (Jitter):** Since the algorithms seek a global energy minimum, the incremental addition of a single new node can cause the entire system to shift or "jitter" substantially, disrupting the user's cognitive map.⁹ Stabilizing these interactive layouts requires specialized extension techniques, such as gradually increasing the influence of newly inserted nodes to ensure a smooth transition.¹⁶
2. **Weak Flow Enforcement:** They do not inherently enforce the unidirectional flow required by a DAG without complex pre-processing or post-processing, making the prerequisite structure less immediately legible than in a layered system.¹⁰

2.3. Constraint-Based Hybrid Layouts (The Recommended Model)

The limitations of pure hierarchical and pure force-directed models necessitate the adoption of a constraint-based hybrid approach. This model leverages the aesthetic quality of the force simulation while enforcing the structural integrity required by the DAG, leading to a stable, explorable map.

Constraint Solving for Stability

For dynamic graphs where nodes are unlocked incrementally, layout stability is paramount. The system must transition smoothly between states. Constraint solvers offer the mechanism for achieving this stability. Toolkits such as Cassowary and its lightweight, high-performance implementation, Kiwi, are designed for efficiently solving systems of linear equalities and inequalities, often optimized for geometric layout within user interfaces.¹⁸ These solvers allow the application to specify mandatory or preferred geometric relationships, such as forcing an element to be "to the left of another" or maintaining alignment.¹⁹

Specialized graph layout libraries have adopted this principle:

- **CoLa.js (Constraint Layout):** This library integrates constraint solving with the force model, resulting in a layout that is significantly more stable (no "jitter") and higher quality than standard D3 force layouts.⁹ CoLa.js allows developers to specify constraints, including alignment, grouping, and non-overlapping nodes, and can automatically generate constraints to provide flow layout for directed graphs.⁹
- **ELK.js:** Positioned as a full-blown, highly configurable layout engine, ELK.js is exceptionally capable for complex, hierarchical structures. It supports dynamic node sizing, sub-flow layout, and comprehensive edge routing, making it an excellent candidate for the highly structured yet aesthetically organic progression map.²¹

The use of a constraint-based solver, such as ELK or CoLa, resolves the primary failure points of dynamic graph visualization. By enforcing DAG flow using layer constraints and preventing jitter through convergence stabilization and incremental updates, the system maintains the critical topological structure required to feel like a persistent "world map."

Implementing Authoring and Pinning Constraints

To allow design teams granular control over the "World Map" aesthetic, the system must support the definition of constraints, including "pinning" critical nodes. Commercial platforms

and advanced libraries like yFiles support considering user-specified preferences on the relative ordering of graph elements and connection points.⁵

In practice, this means the layout algorithm must allow for:

1. **Fixed Positioning (Hard Constraints):** Key "Hero Nodes" (e.g., Biome entry points, seasonal milestones) must be fixed at predetermined coordinates, allowing the constraint solver to calculate the positions of surrounding dependent nodes around these anchors.⁷
2. **Soft Grouping (Aesthetic Constraints):** Specifying preferences such as "these nodes should be clustered" or "these rows should generally align" allows designers to influence the organic clustering provided by the force model, ensuring the visual clusters correspond to thematic areas or "biomes".²²

The ability to specify and iterate on these soft and hard constraints ensures that the final automated visualization meets the intended aesthetic goals, rather than relying solely on the emergent properties of a physical simulation.

2.4. Specialized Layout Patterns: Radial/Concentric Flow

While the constraint-based hybrid model provides the foundational structure, specialized patterns can enhance the visualization of localized progression. Radial layout is particularly effective for visualizing progression radiating from a central point of importance, such as a starting skill or a core ability hub.²³

Radial layout arranges nodes on concentric circles, or layers, where layer count increases outward from the central root node.²³ This visualization naturally emphasizes hierarchical importance (centrality) and directional flow outward. Furthermore, advanced radial layout algorithms can natively support **edge bundling** for non-tree edges.²⁴ This ability is crucial for maintaining clarity in the often-dense starting areas, where many paths diverge from the central root node.

III. Managing Visual Complexity and Information Density

For a map to be considered "large" and "explorable," it must gracefully handle hundreds or thousands of nodes without devolving into visual clutter—the widely acknowledged "hairball"

problem that renders complex networks incomprehensible.²⁶ This necessitates advanced complexity reduction techniques centered around Level-of-Detail (LOD) management.

3.1. Level-of-Detail (LOD) Strategies

LOD techniques are mandatory for graph visualization exceeding a few hundred elements, as human cognitive limits and display device constraints are quickly overwhelmed by overlaps and occlusions.²⁷ The strategy relies on dynamic abstraction.

Hierarchical Clustering and Aggregation

LOD management is built upon an implicit or explicit hierarchy, typically generated algorithmically through clustering based on graph structure or node similarity.²⁹ This process groups neighboring vertices into abstract representations, forming an LOD tree.²⁸ At high zoom levels (low LOD), these groups, or clusters, are represented by visually simplified elements, such as abstract glyphs or implicit surfaces.²⁷ As the user zooms in, the system dynamically transitions, replacing the abstract cluster representations with the individual, detailed nodes. This process allows the user to maintain context of the whole graph while focusing on detailed information in the zoomed area.²⁸

Spatial Indexing for Real-time Performance

Real-time LOD switching and efficient interaction demand high-performance spatial querying. Hierarchical spatial data structures are the established solution. Quadtrees (in 2D space) or R-trees are proven methods for managing multiresolution data, such as terrain height-fields.³¹ These structures efficiently determine which LOD level should be rendered based on the distance between the viewport and the data points.³²

The same spatial indexing structure, particularly R-trees (which are structurally equivalent to hierarchical Axis-Aligned Bounding Box (AABB) trees or Bounding Volume Hierarchies (BVH)³³), must be leveraged to accelerate essential interactions:

- **Collision Detection and Hit-Testing:** In highly interactive graphs, immediate visual feedback (hover states, selection) requires rapid determination of which nodes or edges

intersect the cursor location. A brute-force check of all elements is infeasible for a large graph.³⁵ By utilizing the R-tree structure, complex hit-testing operations can be accelerated through log-time spatial searches, ensuring instantaneous response times critical for maintaining an immersive experience.³⁴

3.2. Clutter Reduction Techniques for Edges (Paths)

The edges, representing the progression paths, are often the primary source of visual clutter in node-link diagrams.

Edge Bundling Implementation

Edge bundling is a crucial technique for reducing clutter by routing similar edge segments into thick visual bundles, effectively revealing high-level relational patterns across the map.³⁶ Advanced bundling methods, sometimes referred to as "metro-line" style, compute edge paths that explicitly minimize length and ink, ensuring individual paths within a bundle are still traceable and do not obscure the nodes.³⁷ This technique is particularly important in the context of large progression maps where multiple nodes might share common prerequisite clusters or future destination points. The radial layout pattern, as noted previously, offers native support for bundling non-tree edges, which is useful for highly interconnected skill groups.²⁴

Visually Implying Directionality

In dense graph visualizations, the overuse of explicit arrows can exacerbate the clutter problem.²⁶ Directionality must be implied subtly yet unambiguously.

Two effective strategies for conveying directional flow are:

1. **Animated Flow:** Leveraging modern graphics capabilities, the direction of progression can be visualized by applying a pulsating or moving gradient animation along the edge path.¹¹ This method, similar to techniques used in cinematic route animations on maps, involves calculating an animation phase (a value between zero and one representing progress along the line) and applying it to a line-gradient property.¹² This provides a

visually dynamic cue for forward momentum without adding geometric clutter.

2. **Visual Differentiation:** Beyond physical movement, link properties can encode information. Using variations in link color, shape, or style allows the designer to differentiate between types of connections (e.g., mandatory prerequisite path vs. optional side quest branch).³⁹

3.3. Dynamic Label Placement and Collision Avoidance

Effective visualization requires that labels are legible and do not overlap nodes or other labels. Since the map supports continuous zooming and panning, the traditional static map labeling problem—which is NP-hard—must be solved interactively.⁴⁰

A high-performance solution must balance speed with consistency, avoiding distracting label "popping".⁴⁰ Solutions fall into two categories:

1. **Pre-computation:** Pre-calculating a preferred label placement and visibility interval for each zoom level minimizes real-time computation during interaction.⁴⁰
2. **Run-time Optimization:** For dynamic maps, real-time algorithms may be employed. Some techniques use a force-directed approach, such as **Simulated Annealing**, where an "energy function" quantifies the extent of overlap between labels. The algorithm then iteratively attempts to minimize this energy by moving or rotating labels based on a weighted calculation.⁴²

For node overlap removal—which directly impacts layout stability—the preferred solution is to integrate a method that minimizes node displacement. The **VPSC (Quadratic Optimization)** overlap removal technique, which is available in many constraint solvers, is superior to simple scaling or heuristic methods like Prism or Voronoi because it removes overlaps while concurrently minimizing the necessary movement of nodes, thus preserving the overall layout stability.⁴⁴

IV. Technical Implementation and Performance Stack

The implementation architecture must prioritize performance to support a "large, explorable, data-driven" map, especially considering the mobile context. This mandates the selection of hardware-accelerated rendering technologies over DOM-based methods.

4.1. Comparative Analysis of Web Rendering Technologies

Selecting the appropriate rendering engine dictates the scalability limit of the application. The three primary web rendering methods offer distinct performance ceilings:

Rendering Technology	Typical Performance Ceiling (Nodes/Edges)	Key Advantage	Key Disadvantage	Best Use Case
SVG	~2,000	Easy DOM interaction, Built-in Accessibility/Styling	Performance degradation with complexity/animation	Static or small to medium graphs, high fidelity interaction ⁴⁵
Canvas 2D	~5,000	Good for moderate dynamic datasets, moderate complexity	Lack of built-in scene graph/object picking	Mid-scale dynamic visualizations, standard charting ⁴⁶
WebGL (e.g., PixiJS)	10,000+	GPU acceleration, High performance for large, complex scenes	Higher complexity, steeper learning curve, harder debugging	Large "World Map" metaphors, real-time animation, high interactivity ⁴⁶

The requirement for a large map visualization that supports continuous zooming, complex LOD transitions, and animated edge flow necessitates performance beyond the 5,000 node limit of Canvas 2D.⁴⁶ **WebGL is the mandatory architecture** for achieving high-fidelity, large-scale graph visualization.⁴⁷ WebGL leverages the computer's GPU, enabling complex

interactive visuals and processing substantial volumes of data points at high speeds.⁴⁷

While WebGL offers unparalleled performance, it introduces significant complexity.⁴⁶ To manage this, the engineering team should adopt a 2D rendering library wrapper built on WebGL, such as **PixiJS**.⁴⁸ These libraries abstract away the complexities of raw WebGL programming, providing a more accessible environment for building high-performance 2D graph scenes.

The final technical stack must combine a robust, constraint-aware layout library (ELK.js or CoLa.js) with a hardware-accelerated rendering engine (WebGL via PixiJS) and a spatial indexing data structure (R-Tree/Quadtree).

4.2. Optimization Strategies for Mobile Performance

Performance degradation, or "jank," is a critical risk when deploying complex graphical applications on mobile devices.⁵⁰ Mobile optimization requires minimizing visual updates and maximizing GPU utilization.

1. **GPU Acceleration for Smoothness:** All complex animations, scrolling, and parallax effects must utilize GPU acceleration to maintain a smooth experience across different screen sizes.⁵¹ Lightweight CSS-based parallax techniques, where background elements move at a different speed than foreground elements to create a sense of depth⁵², should be implemented using GPU-accelerated transforms rather than heavy JavaScript libraries.⁵⁰
2. **Progressive Enhancement and Fallbacks:** Simplified fallback versions of the map (such as lightweight static map images optimized for fast loading and offline use) should be available for environments where interactive maps are not ideal.⁵³
3. **On-Demand API Management:** To minimize network load and resource consumption, API requests that retrieve detailed data (e.g., full information upon clicking a node) must only be initiated on demand, waiting for a user action such as an on-click event.⁵⁴ Furthermore, using exponential backoff is crucial for handling transient API errors, preventing rapid, repeated querying that can exhaust quotas.⁵⁴

V. Interaction Design and Mobile-First UX

The interactive exploration of a large map requires carefully engineered interface patterns

that maintain user orientation and facilitate intuitive interaction, especially on mobile devices.

5.1. Focus + Context Design Patterns

In any large, explorable domain, the user requires solutions that present the current focus (the highly detailed area of the graph) alongside the broader context (the entirety of the map).⁵⁵

A **Minimap** or Overview panel is the standard solution. This panel must be presented in a consistent, non-intrusive location and use visual hierarchy (e.g., high contrast) to clearly delineate the current viewport location within the map's overall extent.⁵⁵

Critically, the design must acknowledge the weaknesses of traditional mobile navigation. Relying on patterns like the Hamburger Menu for core map navigation can lead to the "out of sight, out of mind" issue, obscuring the available options and making it difficult for the user to determine their current position or destination at a glance.⁵⁶ The interactive map itself must serve as the primary navigation canvas, using the minimap purely for orientation confirmation.

5.2. Mobile Gesture Implementation

The interaction model should follow a "Gesture-First" philosophy, utilizing taps, drags, and pinches to add meaningful, intuitive interaction that reduces friction.⁵⁸

Interaction design research indicates that users primarily utilize map gestures based on a **real-world metaphor**.⁵⁹

- When panning, users drag the content itself to reveal previously hidden areas, meaning the viewport moves opposite to the direction of the drag.
- When zooming, users naturally perform gestures that suggest moving a physical object (like a magnifying glass) closer or farther away (Z-plane interaction).⁵⁹

The implementation must strictly adhere to these established mobile map conventions: pinch-to-zoom for scaling and single-finger drag for panning. Because these gestures are handled by the GPU-accelerated WebGL renderer, the movement must be perceived as fluid and continuous, enhancing the immersion.⁵¹

5.3. Accessibility and Keyboard Navigation

Although the primary mode of interaction is visual and gesture-based, the system must support full accessibility for keyboard and screen reader users.⁶⁰ Direct navigation of a visually complex, highly nested, and non-orthogonal graph structure via the standard DOM tab order is functionally impossible.

The solution is to separate the visual layout from the underlying logical hierarchy. The accessibility structure should expose the DAG organization as an invisible, sequential, and logical structure, such as a 'Tree' menu or 'Tab Panel'.⁶¹

Key navigational mappings should include:

- **Tab/Shift + Tab:** Used to navigate between large, interactive elements, such as the overall map canvas, the minimap, and any external controls.⁶²
- **Arrow Keys (Internal Map Focus):** Once the user focuses on the logical structure of the map, the arrow keys should navigate the progression flow. For instance, Up/Down arrows could move between predecessor and successor nodes, or Left/Right arrows could navigate between sibling nodes or expand/collapse nested sub-branches (biomes).⁶¹
- **Enter/Spacebar:** Used to activate a selected node (e.g., opening a detail panel).⁶²

This approach ensures that the complexity of the organic, visual map layout does not interfere with the ability of screen readers to traverse the core progression logic, confirming that the constraint-based layered model is structurally necessary for compliance.

VI. Lifecycle Management and Prototype Recommendations

6.1. Content Lifecycle of Progression Maps

Progression content, especially content tied to ephemeral concepts like "seasons" or limited-time events, must be subjected to a stringent content lifecycle that includes planning, creation, managing, and distribution.⁶³ Crucially, the system requires a formalized strategy for the final stages: preservation and archival.⁶³

In a world map metaphor, simply deleting or removing old, completed, or deprecated nodes (e.g., previous seasonal biomes) destroys the "sense of history" that provides achievement context.⁶⁵

The preferred strategy is: Archival via Aggregation and LOD Transition. Archived or deprecated progression content should not be removed but transitioned into a persistent, low-detail state. The former "biome" area on the map should collapse into a highly simplified representation—a single clustered glyph or an implicit surface representation.²⁷ These preserved structures maintain the spatial integrity of the historical map while greatly reducing rendering complexity and visual clutter. This approach ensures users retain a record of their accomplishments, even if the content is no longer actively maintained or available for new progression.

6.2. Prototype Recommendation Tracks

To mitigate development risk and ensure the final architecture meets the stringent requirements for scale and stability, a phased prototyping approach is recommended. This moves systematically from validating core logical flow to achieving the full high-performance, constrained rendering objective.

Prototype Recommendation Tracks

Track	Primary Goal	Layout Algorithm	Rendering Stack	Complexity/Risk
A: MVP Flowchart	Validating directional flow and basic structure	Dagre (Sugiyama-style) ²¹	SVG (D3.js) ⁴⁵	Low: Fast prototyping, easy debugging.
B: Stable Constraint GVI	Achieving layout stability and enforcing core constraints (pins, layering)	CoLa.js or ELK.js ⁹	Canvas 2D (ECharts, custom D3) ⁴⁶	Medium: Solves jitter/stability problems, moderate complexity.
C:	Full scalability,	ELK.js (Hybrid	WebGL	High: Requires

High-Performance World Map	dynamic LOD, 10k+ elements, 3D aesthetic	Constraint Solver) ²¹	(PixiJS) ⁴⁶	specialized expertise (GPU programming).
-----------------------------------	--	----------------------------------	------------------------	--

Track A: MVP Flowchart (Low Risk): This initial phase focuses on validating the data structure as a clean DAG. Using Dagre, a simple and focused library for laying out directed graphs, coupled with SVG rendering, allows for rapid iteration on the data model and logical structure.²¹ While this phase will fail the large-scale performance and aesthetic requirements, it ensures the hierarchical logic is sound.

Track B: Stable Constraint GVI (Intermediate): This critical phase addresses the primary UX flaw of dynamic graphs: instability. By migrating the layout to a constraint solver like CoLa.js or ELK.js, the team can prototype hard constraints (node pinning) and test incremental layout updates without jitter.⁹ Moving rendering to Canvas 2D⁴⁶ validates performance for mid-scale graphs and allows implementation of basic overlap avoidance (VPSC) and initial Focus + Context UX patterns.

Track C: High-Performance World Map (Target Architecture): This is the mandated target architecture required for supporting a truly large, explorable environment.⁴⁶ It integrates the structural integrity established in Track B (ELK.js) with the mandatory GPU acceleration provided by WebGL (via PixiJS). This phase focuses on advanced features: developing the R-tree spatial index for LOD management and hit-testing³¹, implementing edge bundling, and deploying the animated flow effects.

6.3. Final Technical Roadmap and Tooling Recommendations

The analysis converges on a specific technological triad necessary to achieve a stable, scalable, and aesthetically pleasing progression map:

1. **Constraint Layout Engine: ELK.js** is the most highly recommended engine. It provides comprehensive support for complex hierarchical constraints, robust sub-flow layout, and dynamic node sizing, which is superior to the more focused Dagre library and the slightly less comprehensive constraint set of CoLa.js.²¹
2. **High-Performance Renderer: PixiJS (WebGL 2D)** is the necessary rendering technology. It offers the GPU acceleration required to exceed the 5,000-node ceiling of Canvas, supporting the necessary continuous zooming and high-frame-rate interaction required for an immersive "world map" aesthetic.⁴⁶
3. **Spatial Indexing Structure:** The core computational bottleneck of large interactive graphs (hit-testing and LOD management) must be solved by building an underlying **R-Tree** or Quadtree structure. This structure must be constructed independently of the

layout calculation and used exclusively for accelerated real-time spatial queries.³¹

The project roadmap should therefore proceed with Phase 1 (Data Model Validation/Track A), followed by a focused transition to Phase 2 (Stability and Constraints via ELK.js), and culminating in Phase 3 (Scaling and Fidelity via PixiJS integration and R-Tree spatial acceleration).

Works cited

1. Game Progression and Progression Systems - Game Design Skills, accessed October 8, 2025, <https://gamedesignskills.com/game-design/game-progression/>
2. Keys to Meaningful Skill Trees - GDKeys -, accessed October 8, 2025, <https://gdkeys.com/keys-to-meaningful-skill-trees/>
3. The UX design step up from PoE1 to PoE2 is insane : r/pathofexile - Reddit, accessed October 8, 2025, https://www.reddit.com/r/pathofexile/comments/1gx967r/the_ux_design_step_up_from_poe1_to_poe2_is_insane/
4. Layered graph drawing - Wikipedia, accessed October 8, 2025, https://en.wikipedia.org/wiki/Layered_graph_drawing
5. Layered Graph Layout - yWorks, accessed October 8, 2025, <https://www.yworks.com/pages/layered-graph-layout>
6. Graph Auto-Layout Algorithm | Baeldung on Computer Science, accessed October 8, 2025, <https://www.baeldung.com/cs/graph-auto-layout-algorithm>
7. creating a layout for graph nodes with constraints - Stack Overflow, accessed October 8, 2025, <https://stackoverflow.com/questions/54693558/creating-a-layout-for-graph-nodes-with-constraints>
8. Algorithms for laying out directed graphs? - MathOverflow, accessed October 8, 2025, <https://mathoverflow.net/questions/5253/algorithms-for-laying-out-directed-graphs>
9. cola.js: Constraint-based Layout in the Browser, accessed October 8, 2025, <https://ialab.it.monash.edu/webcola/>
10. Force-directed graph drawing - Wikipedia, accessed October 8, 2025, https://en.wikipedia.org/wiki/Force-directed_graph_drawing
11. Animating Edges - React Flow, accessed October 8, 2025, <https://reactflow.dev/examples/edges/animating-edges>
12. How to create animated route tracking scenes - Mapbox Blog, accessed October 8, 2025, <https://www.mapbox.com/blog/building-cinematic-route-animations-with-mapbox-gl>
13. flowcharts in d3js using dagre-d3 or colajs - Stack Overflow, accessed October 8, 2025, <https://stackoverflow.com/questions/31332711/flowcharts-in-d3js-using-dagre-d3-or-colajs>

14. Graph Layout Performance Comparisons of Force-Directed Algorithms, accessed October 8, 2025, <https://www.ijpe-online.com/EN/10.23940/ijpe.18.01.p8.6776>
15. Graph auto-layout algorithm - Stack Overflow, accessed October 8, 2025, <https://stackoverflow.com/questions/5028433/graph-auto-layout-algorithm>
16. Stable incremental layouts for dynamic graph visualizations - Fraunhofer-Publica, accessed October 8, 2025, <https://publica.fraunhofer.de/entities/publication/bc3339d5-c8d6-40f2-87d2-6aa8ecef245a>
17. STABLE INCREMENTAL LAYOUTS FOR DYNAMIC GRAPH VISUALIZATIONS - IADIS conferences, accessed October 8, 2025, <https://www.iadisportal.org/ijcsis/papers/2013160202.pdf>
18. Cassowary (software) - Wikipedia, accessed October 8, 2025, [https://en.wikipedia.org/wiki/Cassowary_\(software\)](https://en.wikipedia.org/wiki/Cassowary_(software))
19. The Cassowary Linear Arithmetic Constraint Solving Algorithm, accessed October 8, 2025, <https://constraints.cs.washington.edu/solvers/cassowary-tochi.pdf>
20. nucleic/kiwi: Efficient C++ implementation of the Cassowary constraint solving algorithm, accessed October 8, 2025, <https://github.com/nucleic/kiwi>
21. Layouting - Svelte Flow, accessed October 8, 2025, <https://svelteflow.dev/learn/layouting/layouting-libraries>
22. Graph Layout | Tom Sawyer Software, accessed October 8, 2025, <https://www.tomsawyer.com/graph-layout>
23. Radial Layout Style - yWorks, accessed October 8, 2025, https://docs.yworks.com/yfilesflex/doc/dguide-layout/radial_layouter.html
24. Radial Layout | Automatic Graph Layout | yFiles for HTML Documentation, accessed October 8, 2025, https://docs.yworks.com/yfiles-html/dguide/layout/radial_layout.html
25. How can we produce a graph layout with hierarchical edge bundling in Mathematica?, accessed October 8, 2025, <https://mathematica.stackexchange.com/questions/55367/how-can-we-produce-a-graph-layout-with-hierarchical-edge-bundling-in-mathematica>
26. Graphs Beyond the Hairball - eagereyes.org, accessed October 8, 2025, <https://eagereyes.org/techniques/graphs-hairball>
27. Level-of-Detail Visualization of Clustered Graph Layouts, accessed October 8, 2025, <https://graphics.uni-konstanz.de/publikationen/Balzer2007Leveldetailvisualization/Balzer2007Leveldetailvisualization.pdf>
28. An LOD model for graph visualization and its application in web navigation - IBM Research, accessed October 8, 2025, <https://research.ibm.com/publications/an-lod-model-for-graph-visualization-and-its-application-in-web-navigation>
29. Title: A Taxonomy of Glyph Placement Strategies for Multidimensional Data Visualization Author - Worcester Polytechnic Institute, accessed October 8, 2025, https://davis.wpi.edu/xmdv/docs/jinfovis02_glyphpos.pdf

30. Level-of-Detail Visualization of Clustered Graph Layouts - ResearchGate, accessed October 8, 2025, https://www.researchgate.net/publication/30012500_Level-of-Detail_Visualization_of_Clustered_Graph_Layouts
31. Overview of Quadtree-based Terrain Triangulation and Visualization - IFI UZH, accessed October 8, 2025, <https://www.ifi.uzh.ch/dam/jcr:ffffff-82b7-d340-0000-00001b05348a/UCI-ICS-02-01.pdf>
32. ADAPTIVE LEVEL OF DETAIL FOR LARGE TERRAIN VISUALIZATION - International Society for Photogrammetry and Remote Sensing, accessed October 8, 2025, https://www.isprs.org/proceedings/xxxvii/congress/4_pdf/103.pdf
33. What are the disadvantages of R-Trees in collision detection?, accessed October 8, 2025, <https://gamedev.stackexchange.com/questions/105086/what-are-the-disadvantages-of-r-trees-in-collision-detection>
34. Mochi: Fast & Exact Collision Detection - arXiv, accessed October 8, 2025, <https://arxiv.org/html/2402.14801v1>
35. Real-Time_Rendering_4th-Collision_Detection.pdf, accessed October 8, 2025, https://www.realtimerendering.com/Real-Time_Rendering_4th-Collision_Detection.pdf
36. Force-Directed Edge Bundling for Graph Visualization, accessed October 8, 2025, <https://classes.engineering.wustl.edu/cse557/readings/holten-edgebundling.pdf>
37. Edge Routing with Ordered Bundles - Alexander Holroyd, accessed October 8, 2025, <http://aeholroyd.org/papers/bundle.pdf>
38. Building and Analyzing Node-Link Diagrams to Understand Social Networks, accessed October 8, 2025, https://idme-test.ome.ksu.edu/BuildingAnalyzingNodeLinkDiagramsSocialNetworks/BuildingAnalyzingNodeLinkDiagramsSocialNetworks_print.html
39. 5 Link Visualization Styles to Showcase Relationships in Data - Cambridge Intelligence, accessed October 8, 2025, <https://cambridge-intelligence.com/link-visualization-styles/>
40. Dynamic Map Labeling - Computer Science Department, accessed October 8, 2025, <https://www.cs.du.edu/~snarayan/sada/maplabel/classes/docs/mapRef1.pdf>
41. (PDF) Dynamic Map Labeling - ResearchGate, accessed October 8, 2025, https://www.researchgate.net/publication/3411345_Dynamic_Map_Labeling
42. Removing label overlapping from pretty charts | by Adarsh Lilha - Medium, accessed October 8, 2025, <https://medium.com/@adarshlilha/removing-label-overlapping-from-pretty-charts-s-8dd2e3581b71>
43. Dynamic Label Placement in Practice - CEUR-WS.org, accessed October 8, 2025, <https://ceur-ws.org/Vol-1136/paper6.pdf>
44. overlap | Graphviz, accessed October 8, 2025, <https://graphviz.org/docs/attrs/overlap/>
45. [AskJS] do you prefer canvas-based charts or svg-based charts? : r/javascript - Reddit, accessed October 8, 2025,

- https://www.reddit.com/r/javascript/comments/1l3b8sz/askjs_do_you_prefer_canvasbased_charts_or/
46. A Comparison of Javascript Graph / Network Visualisation Libraries - Cylynx, accessed October 8, 2025, <https://www.cylynx.io/blog/a-comparison-of-javascript-graph-network-visualisation-libraries/>
 47. SVG vs. Canvas vs. WebGL: Rendering Choice for Data Visualization - Dev3lop, accessed October 8, 2025, <https://dev3lop.com/svg-vs-canvas-vs-webgl-rendering-choice-for-data-visualization/>
 48. Performance Evaluation of JavaScript Rendering Frameworks - DiVA portal, accessed October 8, 2025, <http://www.diva-portal.org/smash/get/diva2:1411632/FULLTEXT01.pdf>
 49. PixiJS vs react-canvas - Awesome JavaScript - LibHunt, accessed October 8, 2025, <https://js.libhunt.com/compare-pixi-js-vs-react-canvas>
 50. Parallax CSS for Mobile: Design Principles, Performance & Practical Implementation, accessed October 8, 2025, <https://natebal.com/parallax-css/>
 51. The Parallax Effect: Enhancing Depth in Web Design & Digital Media - Garage Farm, accessed October 8, 2025, <https://garagefarm.net/blog/parallax-effect-best-practices-and-examples>
 52. Parallax effect website design: best practices and examples - Justinmind, accessed October 8, 2025, <https://www.justinmind.com/web-design/parallax-effect-website-examples>
 53. Mapbox Maps, accessed October 8, 2025, <https://www.mapbox.com/maps>
 54. Optimization Guide | Google Maps Platform - Google for Developers, accessed October 8, 2025, <https://developers.google.com/maps/optimization-guide>
 55. Focus+context | UXtweak, accessed October 8, 2025, <https://www.uxtweak.com/ux-glossary/focus-context/>
 56. Mobile Navigation Patterns: Pros and Cons - UXPin, accessed October 8, 2025, <https://www.uxpin.com/studio/blog/mobile-navigation-patterns-pros-and-cons/>
 57. Basic Patterns for Mobile Navigation | by Nick Babich - UX Planet, accessed October 8, 2025, <https://uxplanet.org/basic-patterns-for-mobile-navigation-d12a87686efe>
 58. Gesture-First Mobile Design to Increase Onboarding and Retention Beyond Swipe Interaction | by Manoj Kumar - UX Researcher | Sep, 2025 | Medium, accessed October 8, 2025, <https://medium.com/@manoj.uxdesigner/gesture-first-mobile-design-to-increase-onboarding-and-retention-beyond-swipe-interaction-c33eb49d6d48>
 59. User-defined motion gestures for mobile interaction - Google Research, accessed October 8, 2025, <http://research.google.com/pubs/archive/37156.pdf>
 60. Highcharts Accessibility Demos & Examples, accessed October 8, 2025, <https://www.highcharts.com/accessibility/>
 61. Keyboard Accessibility - WebAIM, accessed October 8, 2025, <https://webaim.org/techniques/keyboard/>
 62. How to make charts and graphs more accessible - Pope Tech Blog, accessed

October 8, 2025,

<https://blog.pope.tech/2023/08/31/how-to-make-charts-and-graphs-more-accessible/>

63. 6 Stages of the Digital Content Lifecycle - Acquia, accessed October 8, 2025,

<https://www.acquia.com/blog/digital-content-lifecycle>

64. The 6 stages of the content lifecycle - Digital - Scottish Government Blogs, accessed October 8, 2025,

<https://blogs.gov.scot/digital/2018/08/02/the-6-stages-of-the-content-lifecycle/>

65. What is Lifecycle Mapping? A Comprehensive Guide for UX Designers, accessed October 8, 2025,

<https://juanfernandopacheco.medium.com/what-is-lifecycle-mapping-a-comprehensive-guide-for-ux-designers-d0433276248a>